

Microservices & Spring Boot

Complete Interview Study Guide

Prepared for: Nijitha Mol M

Bannari Amman Institute of Technology

Track: GenC Java Full Stack Engineering

1. What are Microservices?

Interview Answer: Microservices is an architectural style where an application is divided into small, independent services. Each service handles a single business functionality, such as User Service, Order Service, or Payment Service. These services can be developed, deployed, and scaled independently, and they communicate with each other using REST APIs or messaging systems.

Example: Amazon has separate services for User Service, Product Service, Order Service, and Payment Service. If the Payment Service fails, the Product Service can still work.

Key Points:

- Independent services
- Independent deployment
- Better scalability
- Easier maintenance

2. What is Monolithic Architecture?

Interview Answer: Monolithic Architecture is a traditional architecture where the entire application is built as a single project. All modules like User, Product, Order, and Payment are part of one application and are deployed together.

Example: A college management system where Student, Faculty, Library, and Fees modules are all inside one project.

Drawback: If you modify only the Payment module, you must rebuild and redeploy the entire application.

3. Difference Between Monolithic and Microservices

Monolithic	Microservices
Single application	Multiple small services
Single deployment	Independent deployment
Hard to scale	Easy to scale
One database	Separate database (recommended)
Failure affects entire app	Failure affects only one service

Interview Summary: Monolithic applications are easier to start with, but Microservices are better for large-scale applications because they provide better scalability, flexibility, and maintainability.

4. Why do we use Microservices?

Interview Answer: We use Microservices to build large applications in a modular way. Each service is independent, making development, deployment, maintenance, and scaling easier.

Benefits:

- Independent deployment
- Better scalability
- Fault isolation
- Faster development
- Team independence

Example: If only the Product Service receives high traffic, we scale only that service instead of the whole application.

5. Advantages and Disadvantages of Microservices

Advantages:

- Independent deployment
- Easy scalability
- Fault isolation
- Faster development
- Easy maintenance

Disadvantages:

- Complex architecture
- More network communication
- Difficult debugging
- Distributed data management

Interview Tip: Mention 3 advantages and 2 disadvantages.

6. Why is Spring Boot commonly used for Microservices?

Interview Answer: Spring Boot simplifies Microservice development by reducing configuration and providing production-ready features.

Why?

- Auto Configuration
- Embedded Tomcat
- Easy REST API development
- Spring Cloud integration
- Actuator support

Example:

```
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String hello() {
        return "Hello";
    }
}
```

7. What is Spring Boot Auto Configuration?

Interview Answer: Auto Configuration is a feature where Spring Boot automatically configures beans based on the dependencies present in the project. This reduces manual configuration and speeds up development.

Example: After adding the MySQL dependency and datasource properties, Spring Boot automatically creates the DataSource bean.

8. What is Spring Boot Actuator?

Interview Answer: Spring Boot Actuator provides production-ready endpoints to monitor and manage an application.

Common Endpoints:

- /actuator/health
- /actuator/info
- /actuator/metrics

Why? Used to monitor application health, performance, and metrics in production.

9. What are Spring Profiles?

Interview Answer: Spring Profiles allow us to use different configurations for different environments like Development, Testing, and Production.

Example:

- application-dev.properties
- application-test.properties

- application-prod.properties

Activate profile:

```
spring.profiles.active=dev
```

10. Difference Between application.properties and application.yml

application.properties	application.yml
Key-value format	YAML format
Suitable for simple configuration	Better for complex configuration
Less readable for nested data	More readable

Example:

Properties:

```
server.port=8081
```

YAML:

```
server:  
  port: 8081
```

11. What is a REST API?

Interview Answer: REST API is a web service that allows applications to communicate using HTTP methods like GET, POST, PUT, PATCH, and DELETE. It is stateless and commonly exchanges data in JSON format.

Example: GET /users/1

Response:

```
{
  "id": 1,
  "name": "Mohanraj"
}
```

Spring Boot Example:

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
    return userService.getUser(id);
}
```

12. What are HTTP Methods?

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update entire resource
PATCH	Update specific fields
DELETE	Remove data

Spring Boot Example:

```
@GetMapping
@PostMapping
@PutMapping
@PatchMapping
@DeleteMapping
```

13. Difference Between PUT and PATCH

PUT	PATCH
Updates entire resource	Updates only specific fields
Sends complete object	Sends changed fields only
Replaces old data	Modifies existing data

Example:

Current User:

```
{
  "name": "John",
  "email": "john@gmail.com",
  "age": 25
}
```

PUT:

```
{
  "name": "John",
  "email": "new@gmail.com",
  "age": 26
}
```

PATCH:

```
{
  "email": "new@gmail.com"
}
```


14. What is a DTO?

Interview Answer: DTO (Data Transfer Object) is an object used to transfer data between the client and server. It helps hide sensitive information, reduces unnecessary data transfer, and keeps the API independent of the database entity.

Example:

Entity:

```
class User{
    Long id;
    String name;
    String password;
}
```

DTO:

```
class UserDTO{
    Long id;
    String name;
}
```

The password is not exposed to the client.

15. How do you handle exceptions globally in Spring Boot?

Interview Answer: Global Exception Handling is implemented using `@ControllerAdvice` and `@ExceptionHandler`. It allows us to handle exceptions in one place and return consistent error responses.

Example:

```

@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleException(UserNotFoundException ex){
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}

```

Advantages:

- Centralized exception handling
- Cleaner controller code
- Consistent error responses
- Easier maintenance

16. How do Microservices communicate with each other?

Interview Answer: Microservices communicate with each other mainly in two ways:

1. Synchronous Communication

- Uses HTTP/HTTPS.
- The client waits for the response.
- Technologies: RestTemplate, WebClient, OpenFeign.

2. Asynchronous Communication

- Uses Message Brokers.
- The client doesn't wait for the response.
- Technologies: RabbitMQ, Kafka, ActiveMQ.

```

Order Service
  |
  | REST API
  ↓
Payment Service

```

17. Difference Between Synchronous and Asynchronous Communication

Synchronous	Asynchronous
Waits for response	Doesn't wait
HTTP based	Message Queue based
Faster for simple operations	Better for long-running tasks
Tight coupling	Loose coupling

Interview Answer: In synchronous communication, one service waits for another service's response before continuing. In asynchronous communication, services communicate through a message broker, allowing them to work independently without waiting.

18. What is RestTemplate?

Interview Answer: RestTemplate is a Spring class used to make HTTP requests from one microservice to another. It supports GET, POST, PUT, and DELETE.

Note: It is now deprecated in newer Spring versions. WebClient is the recommended replacement.

Example:

```
@RestController
public class OrderController {
    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/order")
    public String getOrder() {
        String response = restTemplate.getForObject(
            "http://localhost:8081/payment",
            String.class);
        return response;
    }
}
```

Create Bean:

```
@Configuration
public class Config {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

19. What is WebClient?

Interview Answer: WebClient is a non-blocking, asynchronous HTTP client introduced in Spring WebFlux. It replaces RestTemplate and provides better performance for microservices.

Example:

```

@Autowired
private WebClient.Builder builder;

public String getPayment() {
    return builder.build()
        .get()
        .uri("http://localhost:8081/payment")
        .retrieve()
        .bodyToMono(String.class)
        .block();
}

```

Advantages:

- Non-blocking and Faster
- Better performance
- Supports Reactive Programming

20. What is OpenFeign Client?

Interview Answer: OpenFeign is a declarative REST client that simplifies communication between microservices. Instead of writing HTTP request code, we define an interface, and Spring generates the implementation automatically.

Enable Feign:

```

@EnableFeignClients
@SpringBootApplication
public class OrderApplication {
}

```

Feign Client:

```

@FeignClient(name = "PAYMENT-SERVICE")
public interface PaymentClient {
    @GetMapping("/payment")
    String getPayment();
}

```

Use It:

```

@RestController
public class OrderController {
    @Autowired
    private PaymentClient paymentClient;

    @GetMapping("/order")
    public String order() {
        return paymentClient.getPayment();
    }
}

```

21. What is Service Discovery?

Interview Answer: Service Discovery is a mechanism that allows microservices to automatically find and communicate with each other without using fixed IP addresses. When a service starts, it registers itself with a Service Registry. Other services can discover it using its service name.

Order Service asks: *"Where is Payment Service?"* → Eureka Server → Returns IP Address

22. What is Eureka Server and Eureka Client?

Interview Answer: Eureka Server acts as a service registry, while Eureka Client registers itself with the registry so other services can discover it using the service name instead of an IP address.

```

      Eureka Server
      /      |      \
Client Client Client
      Order  User   Payment

```

Enable Eureka Server:

```

@EnableEurekaServer
@SpringBootApplication
public class EurekaServerApplication {
}

```

23. What is an API Gateway?

Interview Answer: API Gateway is the single entry point for all client requests. Instead of accessing each microservice directly, the client sends requests to the API Gateway. The Gateway routes requests to the appropriate service.

Advantages: Routing, Authentication, Authorization, Logging, Rate Limiting.

24. Why do we use Spring Cloud Gateway?

Interview Answer: Spring Cloud Gateway is Spring's API Gateway implementation. It routes requests to microservices and provides features like Authentication, Load Balancing, Logging, Rate Limiting, and Request Filtering.

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://USER-SERVICE
          predicates:
            - Path=/users/**
```

25. What is a Circuit Breaker?

Interview Answer: A Circuit Breaker prevents repeated calls to a failed service. If a service is unavailable, the Circuit Breaker stops sending requests temporarily and returns a fallback response.

Benefits: Prevents cascading failures, Improves system stability, Faster recovery.

26. What is Resilience4j?

Interview Answer: Resilience4j is a lightweight Java library used to implement fault tolerance in microservices. It provides Circuit Breaker, Retry, Rate Limiter, Bulkhead, and Time Limiter.

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
public String payment() {
    return paymentClient.getPayment();
}

public String fallback(Exception ex){
    return "Payment Service is currently unavailable.";
}
```

27. What is a Fallback Method?

Interview Answer: A Fallback Method is executed when a service fails or a Circuit Breaker is triggered. Instead of returning an error, it provides an alternative response.

28. How do you secure Microservices using JWT?

Interview Answer: JWT (JSON Web Token) is used for authentication and authorization.

1. User logs in.
2. Server validates credentials and generates a JWT Token.
3. Client sends the token in every request.
4. Server validates the token before processing the request.

```
Header
Authorization: Bearer eyJhbGciOiJIUzI1Ni...
```

29. What is Docker, and why is it used with Microservices?

Interview Answer: Docker is a containerization platform that packages an application along with its dependencies into a container. It ensures the application runs consistently across different environments.

Dockerfile:

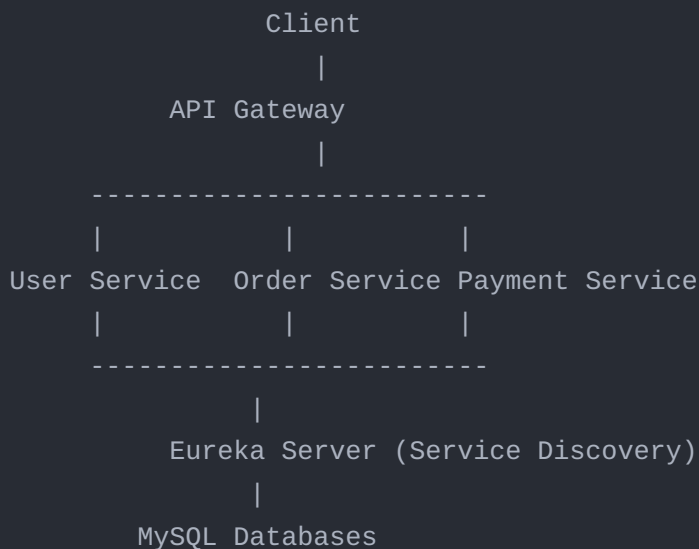
```
FROM eclipse-temurin:21-jdk
COPY target/app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build & Run:


```
docker build -t user-service .
docker run -p 8081:8081 user-service
```

30. Architecture of a Spring Boot Microservices Application

Interview Answer: A typical Spring Boot Microservices architecture consists of:



★ Top 10 Questions to Master

- What are Microservices?
- Monolithic vs Microservices.
- Why Spring Boot for Microservices?
- REST API.
- OpenFeign Client.
- Eureka Server & Eureka Client.
- API Gateway.
- Circuit Breaker & Resilience4j.
- JWT Authentication.
- Complete Microservices Architecture.